



SCHOOL OF COMPUTATION,
INFORMATION AND TECHNOLOGY —
INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Bachelor's Thesis in Informatics

**Adaptation Techniques for using NAS
Methods in the FL Setting**

Max Coetzee





SCHOOL OF COMPUTATION,
INFORMATION AND TECHNOLOGY —
INFORMATICS

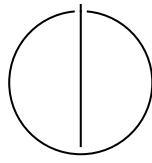
TECHNISCHE UNIVERSITÄT MÜNCHEN

Bachelor's Thesis in Informatics

**Adaptation Techniques for using NAS
Methods in the FL Setting**

Titel

Author:	Max Coetzee
Examiner:	Supervisor
Supervisor:	Advisor
Submission Date:	Submission date



I confirm that this bachelor's thesis is my own work and I have documented all sources and material used.

Munich, Submission date

Max Coetzee

Acknowledgments

Abstract

Both *Neural Architecture Search* (NAS) and *Federated Learning* (FL) have made significant progress independently in the past decade. To benefit from the advantages of NAS methods in FL, researchers have started combining them by using NAS in FL [10] [5] [21].

[TODO: overhaul]

Applying techniques from Neural Architecture Search (NAS) to Federated Learning (FL) has been fruitful (remove) in recent years. The combination was identified as a promising research (remove) direction by [15]. It has yielded methods for finding architectures that deal with the challenges imposed by the FL setting.

Research into NAS has grown rapidly [37] since it was popularized by [54]; consequently, literature on its application to FL has grown. The last survey on NAS applied to FL compared approaches of four papers [53]. Since then, we have identified approximately 50 new papers. This motivates a new systematic survey of the landscape to identify progress and gaps in the literature.

In this thesis, we propose a map of the literature landscape based on the FL challenges they address. We achieve this by systematically evaluating the literature and identifying which challenge it solves.

We refer to the FL challenges described in [54], i.e., non-IID data, limited communication, client heterogeneity, privacy of client data, and break them down into smaller subchallenges — each subchallenge being associated with a pattern in the literature. We include personalized FL [34] as an additional subchallenge that was not originally posited, but has since drawn the community’s attention.

We then analyze how the subchallenges are addressed and focus on the contribution of the used NAS method towards overcoming the subchallenge. For each subchallenge, we keep track of the NAS types used (following [37], [1]) and assess whether the underexplored methods are candidates for future research.

Neural Architecture Search

Contents

Acknowledgments	iv
Abstract	v
1 Introduction	1
2 Background and Related Work	4
2.1 Neural Architecture Search (NAS)	4
2.2 Federated Learning (FL)	5
2.3 NAS in FL	5
3 Method	7
3.1 Reviewed Literature	8
4 Challenges and Solutions for Using NAS in FL	9
4.1 Limited Client Resources	9
4.1.1 Limited Computational Power	10
4.1.2 Limited Networking	13
4.1.3 Client Unreliability	15
4.2 Heterogeneity	16
4.2.1 Hardware Heterogeneity	16
4.2.2 Client Data Heterogeneity	18
4.2.3 Client Networking Capability Heterogeneity	19
4.3 Architecture Search Coordination	19
4.3.1 Coordinating Distributed Search State	20
4.3.2 Coordinating Interdependent Search Decisions	21
4.3.3 Stabilising Distributed Structural Decisions	22
4.3.4 Coordinating Vertically Partitioned Search	23
4.4 Security	24
4.4.1 Challenge 1: Architecture search poisoning	24
4.4.2 Challenge 2: Leaking client data through shared architecture updates	25

Contents

5	Discussion	27
5.1	Principal Findings	27
5.2	Implications for Practice and Research	29
5.3	Limitations	29
5.4	Future Research	30
6	Conclusion	32
	Abbreviations	33
	List of Figures	34
	List of Tables	35
	Bibliography	36

1 Introduction

Deep learning has seen widespread adoption over the last decade and is being applied to an increasingly diverse set of tasks. Applying deep learning traditionally involves a team of deep learning and domain experts who tailor a neural network architecture to a specific task through a lengthy process of trial and error. To automate this process, researchers invented *Neural Architecture Search* (NAS) [7] methods. NAS methods employ diverse strategies to automatically search for a neural network architecture for a given deep learning task within an architecture search space.

Independently, but in parallel to NAS, researchers developed a machine learning approach called *Federated Learning* (FL) [25], which enables training an ML model on data distributed across a set of clients without sharing their data. The model is distributed from the server to the clients, which train the model on their local data. The clients then send the model weight updates to the server, which aggregates them and redistributes them to the clients. This process is repeated for several rounds. FL enhances the privacy of client data and enables training even when clients' data is too large to transfer effectively to a central training site or cannot be shared due to regulatory or privacy constraints.

There has been significant interest in using NAS in FL over the last couple of years, but using NAS methods in FL is not straightforward. Most NAS methods were developed for a centralised setting and rely on assumptions that often do not hold for FL. These assumption discrepancies create several *challenges* for using NAS in FL. For example, in a centralised setting, practitioners can expect a NAS method to run on worker nodes connected via low-latency, high-bandwidth links. However, in FL, a NAS method may run on clients connected via an unreliable, high-latency, low-bandwidth network. This creates a challenge when transferring the weights of large candidate models between clients and the server to coordinate an architecture search. Practitioners need specialised solutions to overcome such challenges, such as only transferring encodings of candidate architectures to clients [32].

The NAS-in-FL literature contains many NAS-in-FL methods that propose various solutions for overcoming the challenges of using NAS in FL [11]. As NAS-in-FL research has progressed, several overarching challenges and solution themes have appeared in the literature. Despite differences in implementation and terminology, solutions are often conceptually similar. For example, different methods use partial-parameter

aggregation or knowledge transfer to coordinate searches involving heterogeneous architectures [rafl_2024, 5, 41].

Earlier reviews classify methods as online or offline and single- or multi-objective, summarise a small set of NAS-in-FL methods within a broader NAS and hyperparameter optimisation review, or discuss NAS as one part of multi-objective FL [53, 17, 9]. More recent work supports comparison by conceptualising a generic NAS-in-FL pipeline, design elements, and method archetypes [11]. While these reviews provide an overview of the types of NAS-in-FL methods, they do not highlight individual challenges of NAS-in-FL methods and the solutions covered in the literature to overcome them. NAS-in-FL research lacks a concept-centric overview of its problem and solution space. Individual studies describe challenges and solutions for particular NAS methods and FL settings using different terminology and levels of abstraction, but they do not reveal which challenges and solutions recur across the literature as overarching concepts.

While themes in challenges and solutions have emerged, they can not be applied universally, since they depend on the FL system characteristics and the used NAS-in-FL method. Furthermore, seemingly related challenges are quite different upon closer inspection and warrant different solutions. A clear overview of the challenge and solution space for NAS-in-FL is needed to make practitioners aware under which circumstance which potential pitfalls occur and make them aware for which circumstances solutions already exist and where work can be done. This leads us to ask the following research question:

Which <adjective placeholder> solutions does the literature propose for addressing the challenges that arise when using NAS in FL?

To tackle our research question, we conduct an extensive systematic literature review. We collect an initial set of relevant literature with a Scopus [6] search and extend it with a snowball search. We first use open coding to identify, from the literature, assumption discrepancies and resulting challenges, when using NAS in FL. Next, we extract FL system characteristics and use axial coding to code the influence of characteristics on the relevance of challenges. Then, we extract techniques through open coding and iteratively refine them to synthesise a coherent set of mutually exclusive and collectively exhaustive techniques. Finally, we use axial coding to map techniques onto challenges they overcome.

Our review aims to synthesise techniques for using NAS in FL from the literature to make finding suitable techniques easy in practice. We make the following three contributions. First, by providing a consolidated overview of the challenges of using NAS in FL, practitioners can grasp which issues they can expect to encounter at a glance. Second, by describing how FL system characteristics influence the relevance of challenges, we enable practitioners to focus on the challenges relevant to them. Third,

by extracting techniques for using NAS in FL and highlighting the challenges they address, we enable practitioners to compare existing techniques for using NAS in FL for their specific set of challenges.

In Chapter 2, we cover the background required for this thesis and related work. In Chapter 3, we describe the method with which we perform our literature review and give an overview of the included literature. In Chapter ??, we highlight the challenges with FedNAS and how they arise from different FL settings. In Chapter ??, we describe the adaptation techniques we synthesised and how they overcome challenges. In Chapter 5, we conduct a discussion about our work. Chapter 6 contains our conclusion.

2 Background and Related Work

Understanding challenges in distributed architecture search requires context about both the learning system and the search method. This chapter provides that context for the applicability judgements in Chapter 4.

2.1 Neural Architecture Search (NAS)

Neural architecture search (NAS) automates the design of a neural network architecture. A NAS method comprises a search space, a search strategy, and a performance-estimation strategy [7, 37]. The search space defines the candidates, including their operations, connections, depth, and width. The search strategy uses observations gathered during the search to select further candidates, while performance estimation approximates how candidates would perform after full training. A search can optimise predictive performance alone or add objectives such as model size, latency, energy consumption, and communication cost.

Search strategies differ in their required work and state. Black-box strategies select discrete candidates and use measured or predicted performance to guide subsequent selections [37]. Evaluating many independently trained candidates is expensive, although the evaluations admit parallel execution.

One-shot strategies represent many candidates within a shared supernet, from which subnetworks inherit weights. Differentiable variants maintain continuous architecture parameters, whereas other variants sample or progressively prune subnetworks. Weight sharing reduces search cost but introduces a large shared state and can make inherited-weight performance an unreliable proxy for independently trained performance [37].

Performance estimation further changes the cost of either strategy. Full training provides high-fidelity evidence at substantial cost; shortened training and predictors trade estimation fidelity for speed [37]. Together, the three NAS components determine which computations clients perform, what search state they store and exchange, and how frequently they evaluate architectures.

2.2 Federated Learning (FL)

Federated learning (FL) enables clients to train a model collaboratively while keeping their raw data local. In a common server-coordinated process, the server selects clients and distributes the global model. Each selected client trains the model locally and returns an update, which the server aggregates for the next communication round [25]. The server consequently receives data-dependent signals; additional privacy mechanisms remain necessary when those signals are sensitive [15].

FL systems vary in scale and client characteristics. Cross-device FL commonly involves many intermittently available mobile or Internet of Things devices, of which only a subset participates in each round. These clients have limited and heterogeneous computation, memory, energy, and wide-area bandwidth [15].

Cross-silo FL usually involves fewer, better-resourced organisations or data centres with stable identities and reliable participation [15]. Its data can be partitioned horizontally, with clients holding different samples, or vertically, with parties holding different features for overlapping samples. In both settings, data quantities and distributions can differ between clients. These settings describe characteristic configurations rather than an exhaustive dichotomy, but establish different expectations about resource constraints, reliability, coordination, and trust.

2.3 NAS in FL

NAS in FL incorporates architecture search into the federated process without centralising client data. Conventional NAS training becomes local client training, while architecture instantiation, evaluation, search-state updates, and selection can occur at the server, at clients, or at both [11]. Clients may exchange model weights together with architecture parameters, candidate encodings, controller state, performance scores, or intermediate representations. The search can produce one global model or client- and group-specific models. Candidate quality also depends on its federated training configuration, coupling architecture search with choices such as local epochs and client participation [11].

The search strategy determines how strongly this additional work affects an FL system. Full-supernet gradient and pruning searches impose high per-round computation and memory demands and require a common supernet representation. Subnet-gradient searches reduce individual client work but must reconcile updates to different parts of the supernet. Population- and controller-based searches place smaller candidates on clients and support parallel evaluation, although repeated evaluations can increase total computation and communication [11]. Surrogate evaluation and weight inheritance

reduce this cost while making candidate rankings depend on proxy evidence [37, 11].

Challenge applicability therefore follows from the combination of FL system and NAS characteristics. Limited resources, dropout, and unequal search participation are especially severe when a one-shot search runs across many cross-device clients. A cross-silo system may make supernet training feasible, while vertically partitioned data introduces end-to-end coordination and intermediate-representation exchanges [15, 11]. Security concerns apply across settings, although the exposed search signal and the parties able to observe it depend on the chosen search.

Solutions inherit the same context dependence: a mechanism that reduces the size of discrete candidate messages does not address a full-supernet memory limit, while aggregation of architecture parameters presumes a compatible shared representation. The challenge–solution analysis therefore considers both the FL system characteristics and the NAS design to delimit where each challenge and solution applies.

3 Method

1. identify initial FL system characteristics 2. identify initial challenges and solutions 3. adjust characteristics to challenges and solutions - retain only challenges and solution which are repeated and have potential => i.e. quite similar to software design patterns [TODO: overhaul to include method for identifying challenges and FL settings] - targeted FL setting can be stated directly, indirectly or must be inferred from experimental setup

We use various techniques from [3] and lean on parts of the methodology of [14] to perform a systematic literature review.

1. **Literature Selection:** We follow the guidelines and flow diagrams provided by PRISMA 2020 [28] for inclusion and exclusion of papers and perform forward and backwards citation searching. We identify 59 papers that present FedNAS methods. Each paper contains one or more FedNAS methods.
2. **Unrefined Adaptation Technique Extraction:** Once the set of included papers is fixed, we perform open coding on each FedNAS method to extract unrefined adaptation techniques. Any modification to a NAS method that is explicitly motivated by the federated setting is treated coded as one unrefined adaptation technique.
3. **Adaptation Techniques Conceptualization:** We iteratively refine and merge unrefined adaptation techniques in an axial coding step to obtain a coherent set of adaptation techniques that are mutually exclusive and collectively exhaustive. Unrefined adaptation techniques with conceptually highly-similar mechanisms are merged into a single representative adaptation technique.
4. **Categorise Adaptation Techniques:** After merging, in a second axial coding step, we cluster adaptation techniques based on a) the FedNAS challenges they address and b) the conceptual similarity of their mechanisms. As a result, we obtain a taxonomy of adaptation techniques.
5. **Discuss FedNAS Challenges for Adaptation Techniques:** We discuss how each adaptation technique works towards, against, or does not affect overcoming each of FedNAS challenges.

6. **Produce Table Overviews:** Finally, we create two tables that practitioners can use to make decisions about FedNAS methods for their use case.

The first table contains a coded vector of effects over the FedNAS challenges for each *FedNAS method*. The effects per FedNAS method are the result of the effects of its adaptation techniques in aggregate. Practitioners can use this table to decide which FedNAS method suits their use case or if their use case would benefit from a new FedNAS method.

The second table contains a coded vector of effects over the FedNAS challenges for each *adaptation technique* based on the prior discussion. If practitioners need to create new FedNAS methods, they can use this table to choose existing adaptation techniques relevant to the set of FedNAS challenges they need to address for their use case.

3.1 Reviewed Literature

add some overview and statistics about the literature

4 Challenges and Solutions for Using NAS in FL

We identified x overarching themes of challenges where the dominant source of the challenge is a FL system characteristic: *Client Resource Constraints*, *Client Heterogeneity*, and *Security*. For these themes the severity and applicability of the challenge varies slightly depending on the search strategy and search space used by the NAS-in-FL (performance evaluation is basically universally training for short rounds) method and the solutions are often specific to one kind of search strategy. Additionally, we grouped challenges where the challenge is closely tied to the architecture search strategy used by a NAS method into another theme *Architecture Search Coordination*.

For each sub-theme we describe concrete challenges relating to that sub-theme and solutions to that challenge from the literature. We start with the most generally applicable solutions to each

- flow chart showing applicability and severity of challenges and solutions at the end of chapter?
- FL system characteristics for which the challenges are applicable/more severe
- NAS types based on NAS in FL and NAS 1000 papers categorisation

4.1 Limited Client Resources

NAS is computationally demanding even in a centralised setting because a search may train and compare many candidate architectures. NAS-Bench-101 made repeated NAS experiments practical through table look-ups by precomputing more than five million trained models for 423,000 unique convolutional architectures, a dataset that required over 100 TPU-years of computation [ying2019nasbench101reproducibleneuralarchitecture]. This scale illustrates the cost that candidate evaluation can impose before FL-specific constraints are considered.

FL compounds this cost because cross-device clients vary in compute performance, memory, bandwidth and availability, and these capabilities are typically lower than those of data-centre nodes [15, 4]. This thematic analysis identified three resource-related sub-themes: limited computation and memory, limited networking and client unreliability.

These constraints are generally less severe in cross-silo FL because its clients are usually fewer, more powerful, more reliable and addressable. This eliminates many cross-device system challenges, although computation or communication can still be the primary bottleneck [15].

4.1.1 Limited Computational Power

Challenge 1.1: Evaluating Candidate Architectures on Constrained Computational Resources

NAS-in-FL can become infeasible when the model that a client must evaluate in one search round exceeds its available memory capacity or the evaluation cannot be completed within the time available for a search round. This concern is central to approaches that target mobile and edge clients [5, 46, 8] where client memory and computational power is rather limited. The problem is particularly acute for Differentiable Supernet-Based strategies that distribute a shared supernet and require clients to update multiple mixed candidate operations locally [dp-fnas_2020, 10, 30, 38]. Other search strategy types usually assign one discrete or generated candidate at a time, but it remains challenging to evaluate even just one candidate model in a feasible amount of time.

Solution 1.1a: Build the search space from inexpensive components. Practitioners can apply this solution to every search strategy type by restricting candidate architectures to inexpensive operations, blocks or modules, depending on the search space type. Prominent examples include depthwise-separable convolutions and mobile-oriented CNN blocks [38, 31]. This makes client-side evaluation cheaper, but may exclude expensive components that increase prediction accuracy.

Solution 1.1b: Prune the search space progressively. During the architecture search, supernet-based strategies can progressively remove weak operations, while black-box optimisation strategies can similarly narrow their search domain after obtaining comparable evidence for discrete candidates [8, 40]. Pruning reduces the alternatives stored and evaluated in later rounds, but leaves the initial cost unchanged and can discard useful operations when applied too early.

Solution 1.1c: Train only one subnet at a time. Supernet-based search strategies can be modified such that clients train and evaluate just a subnet corresponding to a subset of candidate architectures instead of a weighted mixture of all candidate operations, reducing the computational burden on individual clients [8]. The size of the subnet can even be dynamically adjusted according to the computational budget of individual clients, with the most extreme case being just evaluating a single path [divide-and-conquer_fednas_2023, 5, 48, 36, 20]. However, sampling only part of

the supernet in each round can evaluate candidate operations unevenly and thereby bias comparisons between candidate architectures [5, 41].

Challenge 1.2: Managing the Cost of the Federated Search Loop

NAS-in-FL can become impractically expensive when a search strategy estimates the quality of many candidate architectures through separate federated evaluations. Evaluating a candidate may require its distribution, local training and validation, and aggregation over several communication rounds. Repeating this inner FL process across many candidates multiplies the computation required compared with training one fixed architecture [21, 45]. Consequently, the complete search may be infeasible even when clients can evaluate each candidate within their individual computation and memory budgets.

Solution 1.2a: Limit candidate fine-tuning. Candidate-based strategies can initially fine-tune each architecture for only a small number of federated rounds and allocate longer runs only to candidates that perform well [45]. This reduces computation spent on weak candidates, but short evaluations may eliminate architectures that improve only after further training.

Solution 1.2b: Predict candidate performance. - This is considered a standard approach in AutoML (TODO: cite), but requires additional care when using in NAS-in-FL

Candidate-based strategies can train a performance predictor on a small set of evaluated architectures and use it to estimate the quality of the remaining candidates [41, 33]. Clients can also train local predictors on private observations and aggregate only their parameters, sharing performance-estimation knowledge without transmitting locally evaluated architectures [22]. This reduces the number of candidates trained by clients, but its reliability depends on how well the evaluated architectures represent the search space.

Solution 1.2c: Evaluate candidates in parallel. Candidate-based strategies can assign independent candidates to different clients and evaluate them concurrently [32, 52, 33]. When enough clients are available, this reduces elapsed search time but does not reduce the total client computation.

Solution 1.2d: Reuse supernet weights for candidate evaluation. Weight-sharing strategies can train common operation weights and allow each candidate to inherit the subset associated with its architecture [35, 48]. This avoids training every candidate independently from scratch, although inherited-weight performance can rank candidates differently from full training. A short incremental federated fine-tuning stage can provide a higher-fidelity estimate for a smaller set of promising candidates [39].

Challenge 1.3: Post-Search Retraining Duplicates Computation

Many NAS workflows train the selected architecture from scratch after completing the search. In NAS-in-FL, this adds another complete federated optimisation stage after clients have already spent computation on finding and evaluating the architecture. The additional local epochs and aggregation rounds delay the production of a usable model [39, 49].

Solution 1.3a: Reusing Weights Learned during Search

Weight inheritance initialises the selected candidate with weights learned in its supernet, reducing the fine-tuning required after selection. FS-ENAS reports that this changes candidate retraining from minutes to seconds [39].

Solution 1.3b: Producing a Deployable Architecture during Search

One-stage optimisation trains weights while progressively concentrating the architecture distribution or pruning weak operations, so the search terminates with a discrete, trained architecture that can be used without a separate retraining stage [8, 12, 42]. This removes the duplicated federated stage rather than merely shortening it through weight inheritance. Its effectiveness depends on the final path having received sufficient training while it shared weights with competing paths.

Challenge 1.4: Separate Searches for Deployment Targets Multiply Cost

NAS-in-FL systems may need different architectures for several latency or computation budgets. Searching and training independently for every target repeats the complete federated procedure and makes the total cost grow with the number of targets [18]. This cost becomes substantial when one service must support many device classes.

Solution 1.4a: Reusing One Supernet across Deployment Targets

A shared supernet can be trained once through FL and reused to extract architectures for several deployment constraints. SuperFedNAS performs predictor-guided local selection without additional training and reports an elevenfold training-cost reduction for 20 deployment targets [18].

Challenge 1.5: Split-Model Search Leaves Client Computation Idle

In split-model training, different clients or device groups execute consecutive model segments and exchange intermediate activations and gradients. A client can therefore

spend part of each round waiting for the remaining segments to complete, even though it has local computation available. Architecture search inherits this sequential dependency and can leave scarce client resources unused while the search model advances through other partitions [13]. This challenge is specific to NAS-in-FL methods that partition model execution across clients.

Solution 1.5a: Searching Locally during Split-Training Idle Periods

Auxiliary input and output components can temporarily turn a client’s assigned segment into a locally executable search model. During otherwise idle periods, the client samples and trains alternative branches, using the corresponding split-model segment as a teacher for feature distillation [13]. This converts waiting time into search work, but adds local memory and computation and therefore remains suitable only when the client has spare capacity.

4.1.2 Limited Networking

Cross-device clients often communicate with the server over capacity-limited wide-area links. Typical wide-area bandwidths of 5–25 Mbit/s are far below the more than 10 Gbit/s available within data centres [40]. The discussion below assumes a common bandwidth limit; unequal connectivity is treated separately as client heterogeneity.

Challenge 1.6: Search Messages Exceed Client Transfer Budgets

Fixed-model FL exchanges the parameters of one architecture, whereas NAS-in-FL can add an over-parameterised supernet, architecture parameters, or candidate-specific outputs. A supernet contains weights for mutually exclusive operations and can therefore be much larger than any architecture selected from it. Transmitting the complete operation space and its weights can become intractable on a limited link [5, 41]. A client may consequently have enough computation for a local update but insufficient network capacity to receive or return its search state within the communication budget.

Solution 1.6a: Compressing Search Updates

Quantisation represents each update with fewer bits, while sparsification transmits only selected entries. FedRegNAS applies 8-bit stochastic quantisation and retains the largest 20% of entries in its weight and architecture updates before transmission [2]. These operations reduce the message size without changing which architecture variables the search optimises.

Solution 1.6b: Encoding Architecture Choices Compactly

When clients already hold a compatible master model or trained supernet, the server can identify a candidate using a compact selector or genotype instead of transmitting its parameters again. Clients reconstruct the indicated subnet locally and return its evaluation or updated subset of weights [52, 39, 31]. Unlike quantisation or sparsification, this reduces traffic by changing the communicated representation; it requires clients to retain the shared weights and an identical definition of the search space.

Challenge 1.7: Architecture Search Adds Repeated Synchronisation

Differentiable NAS-in-FL updates architecture parameters alongside model weights. Synchronising both variables in every round adds an architecture-update channel that fixed-model FL does not require [10, 2]. Its traffic accumulates throughout the search even when an individual architecture update is small. A long search can therefore exhaust a client’s transfer budget through frequency rather than message size alone.

Solution 1.7a: Synchronising Architecture Parameters Less Frequently

Architecture parameters can be updated locally over several weight-training rounds before they are uploaded. FedRegNAS treats them as slow variables, transmits them every H_α rounds, and stops their transmission after fixing the selected architecture [2]. Model training continues while the additional search channel is used only when needed.

Solution 1.7b: Aggregating Search Updates Hierarchically

A nearby aggregator can combine several clients’ model and architecture updates before forwarding one result to the remote server. Frequent aggregation then uses local links, while the less frequent global exchange traverses the wide-area network [21]. FINCH reports 1.64 GB of traffic at 80% target accuracy on non-IID CIFAR-10, compared with 3.29–4.68 GB for four NAS-in-FL baselines [21].

Solution 1.7c: Parallelising Architecture and Weight Optimisation

Some differentiable formulations make the architecture and weight updates independent within a round, allowing the two computations to proceed in parallel rather than through sequential alternating steps. In vertically partitioned NAS, this reduces the number of synchronised exchanges required to update both variable sets [23]. The solution removes a sequential dependency rather than reducing the size of each message

and therefore applies only when the optimisation formulation permits independent updates.

Challenge 1.8: Communication-Unaware Search Produces Expensive Federated Models

An architecture selected only for predictive performance or local computation can still contain many parameters. FL must transmit those parameters again in every subsequent training round, so the search decision determines the continuing load on each client's link. A model that fits client memory and computation budgets may therefore remain impractical to train federatively under a fixed transfer budget [51]. This cost arises even when every client has the same network capacity.

Solution 1.8a: Optimising the Architecture for Transmitted Model Size

Communication cost can be included directly in the architecture objective. Multi-objective evolutionary search has minimised global error together with the average number of weights uploaded by clients, producing sparse architectures whose updates contain fewer parameters [51]. FedRegNAS similarly includes expected transmitted bytes in its resource objective [2]. The search therefore exposes the accuracy-communication trade-off before an architecture is selected.

4.1.3 Client Unreliability

Challenge 1.9: Client Dropout Interrupts Architecture Search

Cross-device clients can become unavailable because of changing network conditions, battery constraints, or competing local workloads. A search procedure that requires a fixed cohort in every round can then stall or lose candidate updates when selected clients disconnect [48]. This differs from receiving a delayed update: a dropped client provides no update that can subsequently be corrected or down-weighted. Repeated absence can also reduce the representation of that client's data in architecture decisions.

Solution 1.9a: Designing the Search for Partial Participation

The server can select a new random subset of available clients in each round and update the search model from the responses that arrive. Architecture search can consequently continue without waiting for every registered client, while participation can vary between rounds [48]. This solution requires enough available clients to provide a useful

search signal, and random selection alone does not correct systematic unavailability among particular client groups.

Challenge 1.10: Clients Reach Architecture Decisions at Different Times

Clients can optimise architecture variables at different rates because their data distributions and local optimisation trajectories differ. A common stopping round can therefore terminate some local searches prematurely while allowing others to overfit their validation data, for example by increasingly favouring parameter-free operations [24]. The resulting imbalance can reduce the quality of client-specific architectures even when every client remains connected.

Solution 1.10a: Stopping Architecture Updates Independently

After an initial warm-up period, each client can monitor its local architecture and stop updating its architecture variables when a client-specific overfitting criterion is reached. Other clients continue their searches until their own criteria are satisfied, rather than following one synchronised stopping point [24]. This accommodates asynchronous convergence, although the stopping criterion must distinguish genuine convergence from noisy changes caused by a small local validation set.

4.2 Heterogeneity

In FL, clients can differ in hardware, data, and connectivity. These differences create several challenges for NAS.

4.2.1 Hardware Heterogeneity

Challenge 2.1: Architecture Search on Heterogeneous Hardware

Clients participating in NAS-in-FL can vary widely in computation and memory capacity. One client may be able to evaluate a candidate architecture within seconds while another would take minutes to evaluate it. As a result, slow devices delay the architecture search. Some clients may also struggle to fit certain candidate architectures into their memory, so practitioners must adjust the NAS-in-FL to account for this.

Solution 2.1a: Matching Local Search Work to Client Capacity

Resource-aware methods restrict each client to a feasible supernet subspace, subnet, number of cells, or local optimisation budget, so its search work follows its

computation and memory capacity [apons_2023, divide-and-conquer_fednas_2023, supernet_trade-off_fednas_2025, 5, 40, 44].

Solution 2.1b: Grouping Clients by Training Latency

NASFLY profiles client computation and communication costs and forms split-learning groups that minimise mean waiting latency during supernet training [13].

Solution 2.1c: Tolerating Slow or Stale Clients

Straggler-tolerant search removes hard round barriers or down-weights stale client updates, preventing slow clients from blocking or unduly biasing the search [43, 2].

Solution 2.1d: Accounting for Unequal Supernet Exposure

Exposure-aware aggregation averages each supernet parameter only across the clients that trained it, accounting for the unequal operator or width coverage produced by hardware-feasible subsubnets [supernet_trade-off_fednas_2025, 5, 44].

Solution 2.1e: Keeping the Full Search Model on the Server

Server-side methods keep the search space or full supernet off clients: clients receive one sampled candidate, query a cloud supernet for a feasible subnet, or train a lightweight knowledge model from which the server trains the supernet [rafl_2024, 42, 41].

Challenge 2.2: Searching architectures deployable to heterogeneous clients

In NAS-in-FL, the searched architectures are usually trained as well as deployed on the same set of devices. This means in addition to taking client heterogeneity into account at search time, the NAS-in-FL method needs to account for the heterogeneous deployment environment as well. A single architecture tailored to the least capable device can leave the resources of more capable devices unused, while an architecture tailored to the most capable device hardware may be infeasible for deployment on less capable clients [5]. In a cross-device setting, the variance between client hardware capabilities is particularly pronounced.

Solution 2.2a: Per-Client Personalisation

Solution 2.2a: Searching Separate Architectures for Hardware Tiers

Tier-based search assigns clients with similar inference budgets to a common hardware tier and searches one feasible architecture for each tier [5, 47, 44]. This lets stronger devices use more of their capacity than a model sized for the least capable tier.

Solution 2.2c: Training a Shared Supernet for Constraint-Specific Local Selection

After training a weight-sharing supernet, clients select subsubnets or exits that fit their compute and memory limits and rank well on local validation data [18, 41, 50]. This permits architectures to reflect per-client hardware and data heterogeneity while retaining a shared weight space.

Solution 2.2d: Including Hardware Costs in the Search Objective

Hardware-aware search adds a hardware cost, such as measured latency or a MAC budget, to the accuracy objective and selects a candidate within the target limit [47, 20, 44]. Using a different budget or latency profile for each device or hardware group yields architectures tailored to different inference hardware.

4.2.2 Client Data Heterogeneity

Challenge 2.3: Differing Client Data Distributions

Client data can differ in class balance and feature distribution, for example because mobile-phone users have different usage patterns. As clients evaluate candidate architectures on their local data, they may favour different operations or rank the same candidates differently [24, 18]. These conflicting evaluations and training updates make the global search signal inconsistent and can make inherited-weight candidate rankings unreliable [41].

Challenge 2.4: Imbalance Client Data Volumes

Some clients can hold significantly more training data than others. Consequently, estimates of candidate performance from small local datasets can be noisier than those from large datasets. Unequal sample counts can also give data-rich clients more influence over shared weights and architecture decisions, causing the search to favour their data characteristics [10].

4.2.3 Client Networking Capability Heterogeneity

Challenge 2.5: Bias from Overrepresenting Frequently Available and Well-Connected Participants

Network conditions and availability cause some clients to participate more often than others. Clients with slow or unreliable links, low battery, or long candidate runtimes miss more round deadlines, so their data affect fewer architecture decisions [48, 40]. Resource-aware schedulers can compound this imbalance by assigning poorly connected or low-capability clients only small candidates [5]. Each candidate is then trained on a different, non-IID client subset, which couples its apparent quality to the data it received. In weight-sharing searches, this coupling produces conflicting shared-weight updates and unreliable candidate rankings, biasing the search towards frequently available and well-connected clients [41].

Solution 2.5a: Balancing Connectivity and Data Coverage Jointly

Candidate-training groups can be formed using both network constraints and data coverage, preventing connectivity alone from determining which data train each architecture. Group membership is adjusted to give each candidate a more representative data mixture while keeping its communication or completion time feasible. For example, NAFNAS groups clients by bandwidth and class distribution, while FINCH minimises each cluster’s distance from the population distribution under a completion-time limit [27, 21]. These joint assignments make candidate evaluations more comparable because network conditions have less influence on the data seen by each architecture.

Solution 2.5b: Decoupling Client Knowledge from Candidate Assignment

Client knowledge can be aggregated into a common teaching signal before it is used to train different candidate architectures. Each candidate then learns from several clients, rather than only from the devices capable of executing it. For example, HAPFNAS distils averaged predictions from lightweight client models into sampled supernet paths, while PAS uses ensembled client feature maps to guide heterogeneous personalised architectures [41, 24]. Knowledge aggregation therefore transfers information across architecture boundaries without requiring every client to train every candidate.

4.3 Architecture Search Coordination

Centralised NAS maintains the variables and observations that guide a search within one optimisation process. In FL, these responsibilities can be divided between a server

and several clients: clients update model or architecture variables using local data, while the server must turn their results into a coherent search trajectory. The resulting challenges concern which search state is shared, how interdependent search decisions are ordered, and how parties evaluate architectures that span more than one data holder. The computation, communication, heterogeneity, reliability, and security consequences of these mechanisms are discussed in their respective sections.

4.3.1 Coordinating Distributed Search State

Challenge 3.1: Architecture and Model Parameters Form Coupled Global State

Differentiable NAS optimises network weights and continuous architecture parameters together. The architecture parameters determine which candidate operations contribute to the network, while the evidence used to update them depends on the current operation weights. When clients update both variable sets locally, aggregating only the network weights preserves federated model training but discards the structural decisions made during local search. Conversely, combining architecture parameters with unrelated model weights can evaluate structural choices under a different model state from the one that produced them. NAS-in-FL therefore requires an aggregation protocol that reconciles both parts of the shared search state [10, 8, 30].

Solution 3.1a: Aggregating Architecture and Model Parameters Together

Clients can perform local weight and architecture updates and return both variable sets at the end of a federated round. The server applies the same sample-weighted aggregation principle to each set and broadcasts the resulting global architecture parameters together with the corresponding global weights [10, 8, 30]. This maintains one shared differentiable search trajectory across rounds and allows subsequent local updates to start from a mutually consistent state. Parameter-wise aggregation is meaningful only when clients use the same supernet topology and assign the same semantics to every architecture variable. Searches with incompatible client search spaces require a different representation or a separate personalisation mechanism.

Challenge 3.2: Search-Policy Knowledge Can Be Lost between Rounds

Not every search represents its progress as differentiable architecture parameters. Reinforcement-learning approaches can use a controller whose policy is gradually trained to sample promising architectures. If clients are reinitialised when they join a new round, retaining only the sampled model weights loses the controller’s accumulated policy knowledge. Each local search then begins from a cold state even though

previous rounds have already evaluated parts of the search space [29]. The loss can occur despite successful client participation because the coordination protocol omits the information required to continue the search.

Solution 3.2a: Persisting and Redistributing Search-Policy State

The search policy can be made part of the federated state. Clients upload their controller parameters alongside the shared candidate-weight dictionary, after which the server retains controller state and redistributes it for the next round [29]. Newly initialised clients can consequently continue from policy knowledge accumulated in earlier rounds. This principle applies to controller-based searches and, more generally, to search strategies with learned state beyond ordinary model weights. The cited proposal is conceptual and does not empirically establish which server-side rule for choosing or combining controller states is most reliable.

4.3.2 Coordinating Interdependent Search Decisions

Challenge 3.3: Architecture Quality Depends on the Federated Training Configuration

The apparent quality of an architecture depends on the configuration under which it is trained and evaluated. Optimiser choices such as the learning rate interact with architectural depth and gradient propagation, while FL-specific choices such as the number of local epochs and the participating-client fraction change the optimisation trajectory used to obtain a candidate’s fitness [30, 32, 16]. A sequential procedure that fixes one configuration, searches an architecture, and then tunes the configuration can therefore invalidate the comparisons that led to the selected architecture. Coordinating NAS and hyperparameter optimisation is especially important when every candidate is evaluated through a federated training process with its own control variables.

Solution 3.3a: Alternating Architecture and Hyperparameter Optimisation

An alternating procedure can use a hyperparameter-optimisation phase to select a promising training configuration and then run a federated architecture-search phase under that configuration. The results of the architecture phase subsequently inform another hyperparameter decision [30]. Evolutionary search provides a related option in which an individual encodes both an architecture and its training hyperparameters and receives fitness from federated training and client-side validation [32]. Both approaches preserve the dependency between the two decisions during search. Alternation offers

more explicit control over each subproblem, whereas joint evolution can explore cross-dependencies directly at the cost of a larger combined search space.

Solution 3.3b: Including FL Control Variables in the Search Space

The joint search space can include variables that control the federated procedure itself. One black-box formulation represents the number of local training epochs and the fraction of clients activated in each round alongside architecture and conventional hyperparameter choices, and evaluates a complete configuration through decentralised federated training [16]. Candidate comparisons therefore incorporate the FL configuration with which each architecture will operate. This formulation requires the fitness evaluation to capture the complete federated outcome, since a locally favourable control setting need not produce the strongest aggregated model.

4.3.3 Stabilising Distributed Structural Decisions

Challenge 3.4: Structural Decisions Can Destabilise the Search

Federated optimisation repeatedly alternates local updates and server aggregation. During NAS, the same trajectory also changes the model’s structure. Architecture gradients can favour different operations at different depths, and an early preference for parameter-free operations can cause performance collapse before the remaining candidates have been assessed fairly [19]. In progressive searches, committing to an operation changes the active computation graph and the weight-sharing relationships used by subsequent rounds. Making several such decisions without allowing the shared weights to adapt can cause disruption from one decision to distort the evaluation of the next candidate [26].

Solution 3.4a: Regularising Layer-Wise Updates

A client-side alignment term can constrain the relationship between layer gradients and architecture parameters during local differentiable search. The regulariser proposed in [19] measures gradient alignment across layers and incorporates it into the architecture objective, counteracting strongly depth-dependent operation preferences that can prevent architecture convergence. Because the term modifies the structural update before server aggregation, it stabilises the search signal that clients contribute to the shared trajectory. Its influence is controlled by an additional coefficient and therefore introduces a stability hyperparameter that must itself be selected.

Solution 3.4b: Separating Structural Decisions with Warm-Up and Recovery Rounds

Progressive selection can divide the search into explicit phases. Initial warm-up rounds first train the shared supernet weights; the search then fixes one operation at a time using the effect of removing each operation on local validation accuracy. After each structural decision, several recovery rounds update the remaining weights before the next edge is evaluated [26]. Recovery separates the consequences of consecutive structural changes and gives later decisions a more comparable weight state. Once all relevant edges have been resolved, the architecture is fixed and ordinary training continues. This schedule addresses instability within the collective search process; client-specific stopping times are treated separately as a reliability issue.

4.3.4 Coordinating Vertically Partitioned Search

Challenge 3.5: No Party Can Evaluate an End-to-End Candidate Alone

In vertical FL, parties hold different feature sets for the same entities and operate separate portions of a joint model. A local architecture's output becomes useful only after it is combined with the outputs of the other parties, so no feature holder can calculate the complete prediction, loss, or architecture gradient independently. Aggregating architecture parameters is insufficient because the local submodels need not share a topology and their forward computations are complementary [23]. The search protocol must coordinate an end-to-end evaluation while allowing every party to optimise its own portion of the architecture.

Solution 3.5a: Coordinating Local Searches through Intermediate Representations

Each party can retain its own supernet, model weights, and architecture parameters while participating in a split-model forward and backward pass. Feature-holding parties send intermediate outputs to an aggregating party, which computes the joint prediction and loss and returns the corresponding intermediate gradients. Each party then uses the returned gradient to update its local weights or architecture parameters [23]. This provides the end-to-end signal required for coordinated one-shot search without requiring compatible local architectures. Protection of the exchanged representations is a security concern and is therefore addressed separately.

Challenge 3.6: Coordinated Vertical Search Is Restricted to Aligned Samples

The joint forward and backward pass requires records that can be matched across all participating parties. In practice, the intersection of their datasets may contain

far fewer samples than the union of the unaligned local datasets. The coordinated search must then learn both useful representations and architectural choices from a small aligned subset, while most locally available observations cannot contribute to the end-to-end supervised objective [23]. The challenge arises from the sample alignment required to form a collaborative architecture-search signal. Differences between client data distributions remain part of data heterogeneity.

Solution 3.6a: Pretraining Local Search State before Coordinated Search

Before the joint search, each party can perform self-supervised one-shot NAS on all of its local data, updating both its model weights and architecture parameters. The parties then use these locally learned states to initialise supervised vertical NAS on the smaller aligned dataset [23]. Unaligned observations thus contribute to the initial representations and structural preferences even though they cannot produce a joint label-based loss. The supervised collaborative phase remains necessary because independently pretrained local architectures are not guaranteed to be compatible when their outputs are combined.

4.4 Security

4.4.1 Challenge 1: Architecture search poisoning

NAS-in-FL gives participating clients influence over both model weights and architecture parameters, creating two targets for malicious updates [49]. An attacker can poison local data or manipulate its optimisation procedure so that the aggregated updates pursue a backdoor objective and corrupt the architecture search [49]. Weight sharing can spread compromised weights across every candidate that reuses them, distorting comparisons throughout the search space. In an experiment with one attacker among eight clients and a fourfold update-scaling factor, ENAS search became nearly ineffective and SPOS performance declined markedly [49]. The attacker can therefore corrupt a structural decision that persists in the selected architecture.

Solution 1.1: Using Differentiable Search to Suppress Poisoned Operations

SLNAS represents every candidate operation as part of a differentiable mixture. Each client updates both the operation probabilities and their weights. When poisoning makes an operation’s weights oppose or contribute little to the main-task optimisation, later architecture updates reduce that operation’s contribution [49].

Solution 1.2: Restricting the Search Space before Federated Search

GenoSpace restricts the search space before clients participate in the search. It evaluates candidate modules on server-held data and selects a set that jointly balances accuracy, memory, and latency. The resulting constraint on model complexity reduces the tendency to overfit noise introduced by a backdoor trigger [49].

4.4.2 Challenge 2: Leaking client data through shared architecture updates

NAS-in-FL communicates data-dependent search signals in addition to model weights. Differentiable methods share architecture-variable gradients, while other methods transmit discrete architecture graphs, client-specific architecture choices, or scalar rewards [dp-fnas_2020, multi-granular_fednas_2023, 49, 30]. Withholding raw data does not prevent leakage because these signals are computed from local training or validation data [dp-fnas_2020, 30]. An honest-but-curious server or participant may use them to infer properties such as a client’s sample count or class proportions, attempt to reconstruct training examples, or identify the source of an update [multi-granular_fednas_2023, 49]. Architecture search thus exposes information about client data through additional channels that fixed-architecture FL does not require.

Solution 2.1: Applying Differential Privacy to Search Signals

Differential privacy protects data-dependent search messages before clients upload them. DP-FNAS and FEATHERS clip model and architecture gradients and add Gaussian noise, while FEATHERS also perturbs hyperparameter rewards [dp-fnas_2020, 30]. In vertical NAS-in-FL, SS-VFNAS applies the same principle to intermediate activations and backward gradients [23]. Clipping bounds the influence of each sample, while the noise obscures its contribution to the shared signal.

Solution 2.2: Securely Aggregating a Uniform Shared Search

SLNAS keeps a uniform differentiable search representation during collaboration. It applies the same weighted aggregation rule to model weights and architecture parameters, allowing secure aggregation to conceal individual search updates. After the shared search, each client selects and adapts its architecture locally, so the server does not observe its personalised choice [49].

Solution 2.3: Sharing Low-Dimensional Discrete Architecture Encodings

MGFNAS encodes each locally searched architecture as a discrete graph and handles model-parameter training in a separate stage. Its NAPA function aggregates graph entries into probability distributions and samples the next global architecture from them. This low-dimensional search signal makes exact gradient-matching reconstruction underdetermined. In the paper’s seven-layer example, 56 architecture-gradient values constrain 50,176 unknown input pixels [**multi-granular_fednas_2023**].

5 Discussion

- the further the FL system deviates from centralised NAS, the more challenging it is
=> conversely, when NAS-in-FL degenerates to centralised NAS in some aspects it becomes simpler - show how some solutions limit compatibility with others and how they are - the search space trade-off - suprisingly many NAS-in-FL methods do X

5.1 Principal Findings

The findings show that identifying a suitable solution for using NAS in FL requires more than matching a named method to a general challenge. Limited client resources, heterogeneity, architecture-search coordination, and security affect different parts of the search and become relevant under different FL system characteristics. Their effects also overlap. A client’s bandwidth can limit the feasible search state, unequal bandwidth can bias which data inform candidate evaluation, and the resulting structural differences can complicate aggregation. The research problem is therefore best understood as a conditional mapping between a system context, the challenge created in that context, and a compatible solution. This complements method-level classifications and archetypes, which explain how complete NAS-in-FL methods are designed but do not by themselves establish which component is suitable for a particular problem [11].

Resource constraints illustrate why this contextual distinction matters. On weak cross-device clients, local supernet training may exceed compute or memory budgets. Single-path training, progressive pruning, lightweight search spaces, and nested weight sharing reduce this local burden [8, 40, 20]. Candidate-based searches face a different cost because every evaluation can invoke several FL rounds. Limiting fine-tuning, predicting performance, evaluating candidates in parallel, or inheriting supernet weights reduces repeated evaluation, while weight inheritance and one-stage optimisation can avoid a separate retraining stage [45, 41, 39]. A shared supernet can further serve several deployment targets, and split-model searches can use otherwise idle client time [18, 13]. Each solution has different preconditions: a restricted search space requires domain knowledge, parallel evaluation requires enough available clients, and inherited weights provide a cheaper but potentially less faithful ranking than full candidate training.

The communication findings reveal a similar separation between message size, synchronisation frequency, and the communication cost induced by the selected model.

Compression and compact architecture encodings address oversized search messages; infrequent or hierarchical aggregation addresses repeated wide-area synchronisation; and communication-aware objectives prevent the search from producing a model that remains expensive in later FL rounds [2, 21, 51]. Compact encodings are applicable only when clients already share the search space and weights. Hierarchical aggregation presupposes suitable intermediate infrastructure. Consequently, a cross-silo system with capable servers but expensive inter-organisational links may need communication solutions without the compute reductions required by a mobile fleet.

Heterogeneity and unreliability show that feasibility and representativeness must be considered. Matching subnets or workloads to client capacity, grouping clients by latency, tolerating stale updates, and keeping the full search model at the server can prevent weak clients from delaying the search [rafl_2024, 5, 13, 43]. However, clients then train different portions or candidates. Exposure-aware aggregation is needed where parameters overlap, while tier-specific architectures, constraint-specific subnet selection, and hardware-aware objectives address heterogeneous deployment budgets [44, 47, 18]. When availability and connectivity correlate with non-IID data, selecting only fast clients can make candidate comparisons reflect client assignment rather than architecture quality. Unequal sample volumes change estimate reliability and client influence on shared architecture decisions [10]. Forming groups from both network conditions and data coverage, or transferring aggregated knowledge across candidate boundaries, preserves evidence [27, 21, 41]. Partial participation addresses dropout, whereas independent stopping criteria address clients whose architecture variables converge at different times [48, 24]. Treating these as separate contexts prevents a generic “asynchronous” solution from obscuring their different causes. Insufficient capacity bounds what a client can execute, whereas unequal capacity changes whose evidence influences the search; reducing workload addresses the former but may in turn intensify the latter.

Architecture-search coordination depends most directly on the representation of search knowledge. Differentiable searches with a common supernet can aggregate model and architecture parameters together, but this requires identical meanings for every aggregated variable [10, 8]. Controller-based searches instead have to persist their learned policy, while searches whose architecture quality depends on FL hyperparameters must alternate or jointly optimise these coupled decisions [29, 30, 32]. Regularisation or warm-up and recovery rounds can stabilise structural decisions, depending on whether instability arises in layer-wise gradients or progressive pruning [19, 26]. Vertical FL forms a distinct context because no feature holder can evaluate an end-to-end candidate alone. Exchanging intermediate representations supplies a joint optimisation signal, and self-supervised local pretraining makes unaligned samples useful before coordinated search on the aligned subset [23].

Security completes the mapping by distinguishing integrity from confidentiality. A malicious client can influence persistent architecture decisions, for which suppressing poisoned operations or restricting the search space provides reported protection [49]. An honest-but-curious party presents another problem: architecture gradients, discrete choices, rewards, and intermediate representations remain data-dependent even though raw data stay local. Differential privacy, secure aggregation of a uniform shared search, and low-dimensional discrete encodings protect different observable search signals [dp-fnas_2020, multi-granular_fednas_2023, 30]. Their applicability follows the threat model and search representation, and privacy noise or a restricted representation can weaken the evidence guiding architecture selection.

5.2 Implications for Practice and Research

For practitioners, the findings support a diagnostic sequence. The intended system should first be characterised by FL partitioning, client compute and memory, link capacity, participation, data distributions, trust assumptions, and deployment budgets. The search representation can then be examined to locate the constrained or exposed state. Only after these conditions are explicit can the relevant solutions be combined. For example, capacity-specific subnets may require exposure-aware aggregation, representative client grouping, compact communication, and protection of architecture updates. Selecting one component without its dependencies can transfer cost, bias, or exposure to another part of the search. This makes the map useful before implementation because it reveals which assumptions must hold for a solution to remain operationally feasible.

For research, this synthesis suggests that solution configurations are a more informative unit of comparison than complete NAS-in-FL methods. It provides a vocabulary for distinguishing where a federated condition affects the search process, the evidence used to guide it, or the architecture eventually deployed. It also exposes dependencies between solution components that method-level comparisons can conceal. The mapping therefore supports condition-specific comparison rather than a universal ranking of solutions. Its main implication is that the applicable context should form part of every challenge–solution claim.

5.3 Limitations

The literature search limits the coverage of the synthesis. It uses an English-language Scopus query and one iteration of backward and forward citation searching. Publications using different terminology, indexed elsewhere, or published after the search

may therefore be absent. Excluding surveys and publications without enough methodological detail improves the evidential basis for coding, but it can omit challenges or solutions reported only at a high level. The identified concepts should consequently be understood as a synthesis of the reviewed corpus rather than an exhaustive account of every possible NAS-in-FL design.

The thematic analysis introduces interpretative limitations. The coding captures how authors describe and motivate challenges and solutions, and the applicability of a solution sometimes has to be inferred from the reported FL configuration or experimental setup. Decisions to merge conceptually similar mechanisms and separate superficially similar challenges are grounded in the coded extracts, but alternative researchers could draw theme boundaries differently. Repeated comparison with the complete corpus provides an audit trail for these decisions without eliminating the researcher judgement inherent in familiarisation, coding, and theme refinement [using_thematic_analysis_2006].

Finally, a relationship in the mapping indicates that the literature presents a solution as addressing a challenge under particular conditions. It does not establish the size of that effect or prove that the solution will transfer to another system. The reviewed studies differ in datasets, search spaces, client populations, hardware, resource models, and evaluation budgets. These differences prevent a quantitative comparison across solutions and make publication frequency unsuitable as a measure of effectiveness, severity, or practical prevalence. The findings can therefore guide conditional selection and hypothesis formation, but they cannot guarantee performance in an unstudied FL context.

5.4 Future Research

Future research should test the challenge–solution relationships under controlled and comparable conditions. A shared evaluation framework could vary client compute and memory, bandwidth, data distributions, participation, and deployment budgets while holding the learning task and search space constant. Evaluations should distinguish search cost from the cost of training and deploying the selected architecture. They should also report wall-clock time, transferred bytes, candidate-ranking fidelity, participating-client coverage, final-model quality, and the assumed threat model. These measurements would reveal which system characteristic explains a solution’s benefit and where its reported applicability ceases.

The interactions between solutions require particular attention. Resource-aware workload assignment can keep weak clients involved while changing their exposure to supernet operations; compact or noisy search signals can reduce communication

or leakage while weakening architecture evidence; and personalised deployment can require a compatible aggregation mechanism. Factorial experiments could evaluate these components individually and in combination, including configurations that address several challenges simultaneously. Such comparisons would show whether their benefits accumulate, interfere, or shift costs to another part of the search.

Further studies should evaluate NAS in FL under combinations of conditions that occur together in operational systems. Dropout, stale updates, unequal connectivity, and non-IID data can jointly determine which clients influence architecture selection. Security evaluations should likewise consider both the predictive model and the integrity of the structural decision, while stating which search artefacts an adversary can observe or manipulate. Evidence from cross-device, cross-silo, and vertical FL deployments would test whether relationships derived largely from individual experimental settings remain useful at operational scale.

Future reviews can extend the database coverage, repeat citation searching, involve multiple coders, and validate the challenge definitions with practitioners responsible for deployed FL systems. A common reporting scheme for client characteristics, search-time resources, deployment constraints, participation, architecture overlap, and security assumptions would make new proposals easier to compare. With stronger comparative evidence, the present mapping could support a decision aid that recommends solution configurations for an explicitly described FL system.

6 Conclusion

- the amount of possible knobs to turn in NAS-in-FL is astronomical and the literature has only investigated a small subset of them - NAS in FL trails behind NAS and FL - provide some clear direction for future research

Abbreviations

List of Figures

List of Tables

Bibliography

- [1] S. S. P. Avval, N. D. Eskue, R. M. Groves, and V. Yaghoubi. “Systematic review on neural architecture search.” In: *Artificial Intelligence Review* 58.3 (Jan. 6, 2025), p. 73. ISSN: 1573-7462. DOI: 10.1007/s10462-024-11058-w.
- [2] Z. Chen, H. Zhang, S. Wang, and J. Chen. “FedRegNAS: Regime-Aware Federated Neural Architecture Search for Privacy-Preserving Stock Price Forecasting.” In: *Electronics* 14.24 (2025). DOI: 10.3390/electronics14244902.
- [3] J. Corbin and A. Strauss. *Basics of qualitative research*. Core textbook v. 14. SAGE Publications, 2015. ISBN: 9781412997461.
- [4] K. Daly, H. Eichner, P. Kairouz, H. B. McMahan, D. Ramage, and Z. Xu. “Federated learning in practice: Reflections and projections.” In: *2024 IEEE 6th international conference on trust, privacy and security in intelligent systems, and applications (TPS-ISA)*. 2024, pp. 148–156. DOI: 10.1109/TPS-ISA62245.2024.00026.
- [5] L. Dudziak, S. Laskaridis, and J. Fernandez-Marques. *FedorAS: Federated architecture search under system heterogeneity*. 2022. arXiv: 2206.11239 [cs.LG].
- [6] Elsevier. *Scopus*. Abstract and citation database. Accessed 2026-01-17. 2026.
- [7] T. Elsken, J. H. Metzen, and F. Hutter. “Neural architecture search: A survey.” In: *Journal of Machine Learning Research* 20.55 (2019), pp. 1–21.
- [8] A. Garg, A. K. Saha, and D. Dutta. *Direct federated neural architecture search*. 2020. arXiv: 2010.06223 [cs.LG].
- [9] M. Hartmann, G. Danoy, and P. Bouvry. *Multi-objective methods in Federated Learning: A survey and taxonomy*. 2025. arXiv: 2502.03108 [cs.LG].
- [10] C. He, M. Annavaram, and S. Avestimehr. *Towards Non-I.I.D. and invisible data with FedNAS: Federated deep learning via neural architecture search*. 2021. arXiv: 2004.08546 [cs.LG].
- [11] N. Henze, S. Rank, D. Deng, M. Lindauer, A. Sunyaev, and N. Kannengießer. “Neural Architecture Search in Federated Learning: A Unified Design Concept.” In: *TechRxiv* (2026). DOI: 10.36227/techrxiv.176964091.16274890/v1.
- [12] X. Huang and J. Gao. *Federated neural architecture search with model-agnostic meta learning*. 2025. arXiv: 2504.06457 [cs.LG].

- [13] C. Huo, J. Jia, T. Deng, M. Dong, Z. Yu, and D. Yuan. "NASFLY: On-device split federated learning with neural architecture search." In: *2024 IEEE international symposium on parallel and distributed processing with applications (ISPA)*. 2024, pp. 2184–2190. doi: 10.1109/ISPA63168.2024.00298.
- [14] D. Jin, N. Kannengießer, S. Rank, and A. Sunyaev. "Collaborative distributed machine learning." In: 57.4 (Dec. 2024). issn: 0360-0300. doi: 10.1145/3704807.
- [15] P. Kairouz, H. B. McMahan, B. Avent, A. Bellet, M. Bennis, A. N. Bhagoji, K. Bonawitz, Z. Charles, G. Cormode, R. Cummings, R. G. L. D'Oliveira, H. Eichner, S. E. Rouayheb, D. Evans, J. Gardner, Z. Garrett, A. Gascón, B. Ghazi, P. B. Gibbons, M. Gruteser, Z. Harchaoui, C. He, L. He, Z. Huo, B. Hutchinson, J. Hsu, M. Jaggi, T. Javidi, G. Joshi, M. Khodak, J. Konečný, A. Korolova, F. Koushanfar, S. Koyejo, T. Lepoint, Y. Liu, P. Mittal, M. Mohri, R. Nock, A. Özgür, R. Pagh, M. Raykova, H. Qi, D. Ramage, R. Raskar, D. Song, W. Song, S. U. Stich, Z. Sun, A. T. Suresh, F. Tramèr, P. Vepakomma, J. Wang, L. Xiong, Z. Xu, Q. Yang, F. X. Yu, H. Yu, and S. Zhao. *Advances and Open Problems in Federated Learning*. 2021. arXiv: 1912.04977 [cs.LG].
- [16] S. Khan, F. Nosheen, S. S. A. Naqvi, H. Jamil, M. Faseeh, M. A. Khan, and D.-H. Kim. "Bilevel Hyperparameter Optimization and Neural Architecture Search for Enhanced Breast Cancer Detection in Smart Hospitals Interconnected With Decentralized Federated Learning Environment." In: *IEEE Access* 12 (2024), pp. 63618–63628. doi: 10.1109/ACCESS.2024.3392572.
- [17] S. Khan, A. Rizwan, A. N. Khan, M. Ali, R. Ahmed, and D. H. Kim. "A multi-perspective revisit to the optimization methods of Neural Architecture Search and Hyper-parameter optimization for non-federated and federated learning environments." In: *Computers and Electrical Engineering* 110 (2023), p. 108867. issn: 0045-7906. doi: <https://doi.org/10.1016/j.compeleceng.2023.108867>.
- [18] A. Khare, A. Agrawal, A. Annavajjala, P. Behnam, M. Lee, H. Latapie, and A. Tumanov. *SuperFedNAS: Cost-efficient federated neural architecture search for on-device inference*. 2024. arXiv: 2301.10879 [cs.LG].
- [19] J. Li, S. Wang, R. Yang, M. Gong, Z. Hu, N. Zhang, K. Sheng, and Y. Zhou. "Toward Federated Customized Neural Architecture Search for Remote Sensing Scene Classification." In: *IEEE Transactions on Geoscience and Remote Sensing* 63 (2025), pp. 1–12. doi: 10.1109/TGRS.2025.3537085.
- [20] Y. Li, R. Yao, D. Qin, and Y. Wang. "Lightweight federated learning for on-device non-intrusive load monitoring." In: *IEEE Transactions on Smart Grid* 16.2 (2025), pp. 1950–1961. doi: 10.1109/TSG.2024.3482363.

- [21] J. Liu, J. Yan, H. Xu, Z. Wang, J. Huang, and Y. Xu. "Finch: Enhancing federated learning with hierarchical neural architecture search." In: *IEEE Transactions on Mobile Computing* 23.5 (2024), pp. 6012–6026. doi: 10.1109/TMC.2023.3315451.
- [22] S. Liu, X. Wang, and Y. Jin. "Federated Bayesian Optimization for Privacy-Preserving Neural Architecture Search." In: *2023 IEEE Congress on Evolutionary Computation (CEC)*. 2023, pp. 1–8. doi: 10.1109/CEC53210.2023.10254155.
- [23] Y. Liu, X. Liang, J. Luo, Y. He, T. Chen, Q. Yao, and Q. Yang. "Cross-Silo Federated Neural Architecture Search for Heterogeneous and Cooperative Systems." In: *Federated and Transfer Learning*. Ed. by R. Razavi-Far, B. Wang, M. E. Taylor, and Q. Yang. Cham: Springer International Publishing, 2023, pp. 57–86. ISBN: 978-3-031-11748-0. doi: 10.1007/978-3-031-11748-0_4.
- [24] Y. Liu, S. Guo, J. Zhang, Z. Hong, Y. Zhan, and Q. Zhou. "Collaborative Neural Architecture Search for Personalized Federated Learning." In: *IEEE Transactions on Computers* 74.1 (2025), pp. 250–262. doi: 10.1109/TC.2024.3477945.
- [25] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas. "Communication-efficient learning of deep networks from decentralized data." In: *Proceedings of the 20th international conference on artificial intelligence and statistics*. Ed. by S. Aarti and Z. Jerry. Vol. 54. Proceedings of machine learning research. PMLR, Apr. 2017, pp. 1273–1282.
- [26] E. Mushtaq, C. He, J. Ding, and S. Avestimehr. *SPIDER: Searching personalized neural architecture for federated learning*. 2021. arXiv: 2112.13939 [cs.LG].
- [27] G. Öcal and A. Özgövde. "Network-aware federated neural architecture search." In: *Future Generation Computer Systems* 162 (2025), p. 107475. ISSN: 0167-739X. doi: <https://doi.org/10.1016/j.future.2024.07.053>.
- [28] M. J. Page, J. E. McKenzie, P. M. Bossuyt, I. Boutron, T. C. Hoffmann, C. D. Mulrow, L. Shamseer, J. M. Tetzlaff, E. A. Akl, S. E. Brennan, R. Chou, J. Glanville, J. M. Grimshaw, A. Hróbjartsson, M. M. Lalu, T. Li, E. W. Loder, E. Mayo-Wilson, S. McDonald, L. A. McGuinness, L. A. Stewart, J. Thomas, A. C. Tricco, V. A. Welch, P. Whiting, and D. Moher. "The PRISMA 2020 statement: an updated guideline for reporting systematic reviews." In: *BMJ* 372 (2021). doi: 10.1136/bmj.n71. eprint: <https://www.bmj.com/content/372/bmj.n71.full.pdf>.
- [29] T. Ramírez-Gordillo, H. Mora, F. A. Pujol, and A. Maciá-Lillo. "Federated Neural Architecture Search for Efficient and Privacy-Preserving Model Training." In: *2024 IEEE International Conference on Smart Internet of Things (SmartIoT)*. 2024, pp. 129–136. doi: 10.1109/SmartIoT62235.2024.00028.

- [30] J. Seng, P. Prasad, M. Mundt, D. S. Dhimi, and K. Kersting. *FEATHERS: Federated Architecture and Hyperparameter Search*. 2023. arXiv: 2206.12342 [cs.LG].
- [31] J.-M. Shao, G.-Q. Zeng, K.-D. Lu, G.-G. Geng, and J. Weng. "Automated federated learning for intrusion detection of industrial control systems based on evolutionary neural architecture search." In: *Computers & Security* 143 (2024), p. 103910. DOI: 10.1016/j.cose.2024.103910.
- [32] R. E. Shawi. "AgingFedNAS: Aging evolution federated deep learning for architecture and hyperparameter search." In: *Advances in knowledge discovery and data mining*. Ed. by X. Wu, M. Spiliopoulou, C. Wang, V. Kumar, L. Cao, Y. Wu, Y. Yao, and Z. Wu. Singapore: Springer Nature Singapore, 2025, pp. 107–119. ISBN: 978-981-96-8173-0.
- [33] J. Su, C. Tang, and Z. Liu. "A Prediction Optimization Method with Federated Learning and Neural Architecture Search for Distributed Renewable Energy Sources." In: *Distributed Generation & Alternative Energy Journal* 40.2 (2025), pp. 213–238. DOI: 10.13052/dgaej2156-3306.4021.
- [34] A. Z. Tan, H. Yu, L. Cui, and Q. Yang. "Towards personalized federated learning." In: *IEEE Transactions on Neural Networks and Learning Systems* 34.12 (2023), pp. 9587–9603. DOI: 10.1109/TNNLS.2022.3160699.
- [35] C. Wang, B. Chen, G. Li, and H. Wang. "Automated Graph Neural Network Search under Federated Learning Framework." In: *IEEE Transactions on Knowledge and Data Engineering* 35.10 (2023), pp. 9959–9972. DOI: 10.1109/TKDE.2023.3250264.
- [36] X. Wei, G. Chen, C. Yang, H. Zhao, C. Wang, and H. Yue. "EFNAS: Efficient federated neural architecture search across AIoT devices." In: *2024 international joint conference on neural networks (IJCNN)*. 2024, pp. 1–8. DOI: 10.1109/IJCNN60899.2024.10650653.
- [37] C. White, M. Safari, R. Sukthankar, B. Ru, T. Elsken, A. Zela, D. Dey, and F. Hutter. *Neural architecture search: Insights from 1000 papers*. 2023. arXiv: 2301.08727 [cs.LG].
- [38] R. Wu, C. Li, J. Zou, and S. Wang. "FedAutoMRI: Federated neural architecture search for MR image reconstruction." In: *Medical Image Computing and Computer Assisted Intervention – MICCAI 2023 Workshops*. Berlin, Heidelberg: Springer, 2023, pp. 347–356. DOI: 10.1007/978-3-031-47401-9_33.
- [39] Y. Wu, H. Fan, W. Ying, Z. Zhou, Q. Zheng, and J. Zhang. "Federated Neural Architecture Search with Hierarchical Progressive Acceleration for Medical Image Segmentation." In: *Advances in Swarm Intelligence*. Ed. by Y. Tan and Y. Shi. Singapore: Springer Nature Singapore, 2024, pp. 112–123. ISBN: 978-981-97-7184-4.

- [40] J. Yan, J. Liu, H. Xu, Z. Wang, and C. Qiao. "Peaches: Personalized federated learning with neural architecture search in edge computing." In: *IEEE Transactions on Mobile Computing* 23.11 (2024), pp. 10296–10312. doi: 10.1109/TMC.2024.3373506.
- [41] A. Yang and Y. Liu. "Heterogeneity-aware personalized federated neural architecture search." In: *Entropy* 27.7 (2025). issn: 1099-4300. doi: 10.3390/e27070759.
- [42] D. Yao and B. Li. "PerFedRLNAS: One-for-all personalized federated neural architecture search." In: *Proceedings of the AAAI Conference on Artificial Intelligence* 38.15 (Mar. 2024), pp. 16398–16406. doi: 10.1609/aaai.v38i15.29576.
- [43] D. Yao, L. Wang, J. Xu, L. Xiang, S. Shao, Y. Chen, and Y. Tong. "Federated model search via reinforcement learning." In: *2021 IEEE 41st international conference on distributed computing systems (ICDCS)*. 2021, pp. 830–840. doi: 10.1109/ICDCS51616.2021.00084.
- [44] W. Ying, C. Wang, Y. Wu, X. Luo, Z. Wen, and H. Zhang. "FGFL: Fine-grained federated learning based on neural architecture search for heterogeneous clients." In: *Advances in Swarm Intelligence*. Singapore: Springer Nature Singapore, 2024, pp. 99–111.
- [45] J. Yuan, M. Xu, Y. Zhao, K. Bian, G. Huang, X. Liu, and S. Wang. *Federated neural architecture search*. 2022. arXiv: 2002.06352 [cs.LG].
- [46] J. Yuan, M. Xu, Y. Zhao, K. Bian, G. Huang, X. Liu, and S. Wang. "Resource-Aware Federated Neural Architecture Search over Heterogeneous Mobile Devices." In: *IEEE Transactions on Big Data* (2022). doi: 10.1109/TBDATA.2022.3227403.
- [47] C. Zhang, X. Yuan, Q. Zhang, G. Zhu, L. Cheng, and N. Zhang. "Toward tailored models on private AIoT devices: Federated direct neural architecture search." In: *IEEE Internet of Things Journal* 9.18 (2022), pp. 17309–17322. doi: 10.1109/JIOT.2022.3154605.
- [48] F. Zhang, J. Ge, C. Wong, S. Zhang, C. Li, and B. Luo. "Optimizing Federated Edge Learning on Non-IID Data via Neural Architecture Search." In: *2021 IEEE Global Communications Conference (GLOBECOM)*. 2021, pp. 1–6. doi: 10.1109/GLOBECOM46510.2021.9685909.
- [49] F. Zhang, M. Li, J. Ge, F. Tang, S. Zhang, J. Wu, and B. Luo. "Privacy-Preserving Federated Neural Architecture Search with Enhanced Robustness for Edge Computing." In: *IEEE Transactions on Mobile Computing* 24.3 (2025), pp. 2234–2252. doi: 10.1109/TMC.2024.3490835.

- [50] Z. Zhang, Z. Liu, Y. Zhao, C. Qiu, C. Zhang, and X. Wang. “ENASFL: A federated neural architecture search scheme for heterogeneous deep models in distributed edge computing systems.” In: *IEEE Transactions on Network Science and Engineering* 11.3 (2024), pp. 2610–2620. DOI: 10.1109/TNSE.2023.3344850.
- [51] H. Zhu and Y. Jin. “Multi-Objective Evolutionary Federated Learning.” In: *IEEE Transactions on Neural Networks and Learning Systems* 31.4 (2020), pp. 1310–1322. DOI: 10.1109/TNNLS.2019.2919699.
- [52] H. Zhu and Y. Jin. *Real-time federated evolutionary neural architecture search*. 2020. arXiv: 2003.02793 [cs.LG].
- [53] H. Zhu, H. Zhang, and Y. Jin. “From federated learning to federated neural architecture search: a survey.” In: *Complex and Intelligent Systems* 7.2 (Apr. 2021), pp. 639–657. ISSN: 2198-6053. DOI: 10.1007/s40747-020-00247-z.
- [54] B. Zoph and Q. V. Le. *Neural architecture search with reinforcement learning*. 2017. arXiv: 1611.01578 [cs.LG].